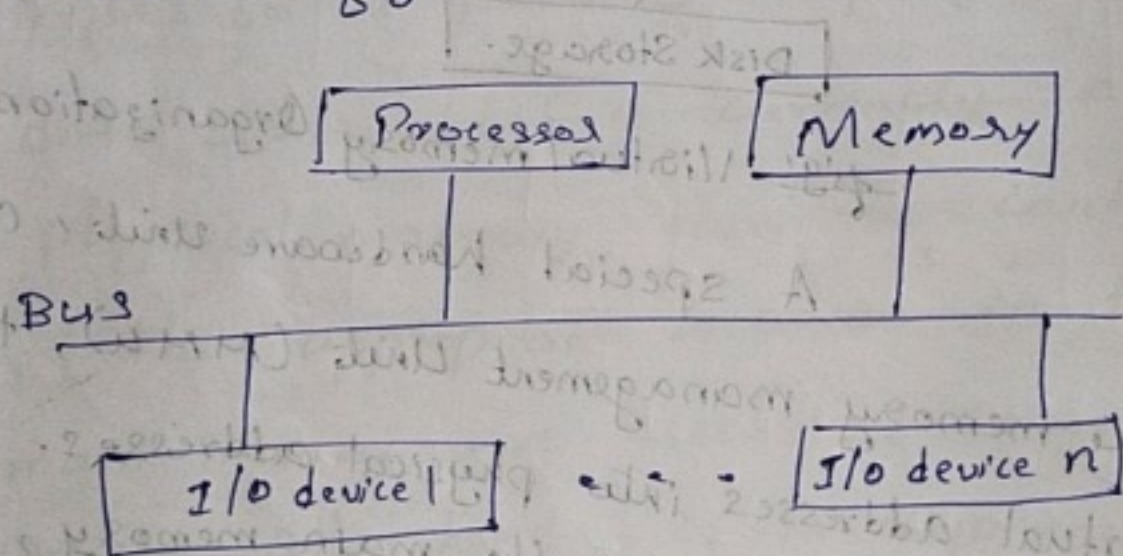


Module - II

Input / Output Organization

I. Accessing I/O Devices

A Simple Arrangement to connect I/O devices to a computer is to use a single bus arrangement, as shown in figure.



The bus enables all the devices connected to it to exchange information. It consists of three sets of lines to carry address, data and control signals. Each I/O device is assigned with a unique set of addresses. When the processor places a particular address on the address lines, the device that recognizes

this address responds to the command issued on the control lines. The processor requests either a read or write operation, and the requested data are transferred over the data lines.

(1) Memory Mapped I/O

When I/O device and memory share the same address space, the arrangement is called Memory mapped I/O. With memory mapped I/O, any machine instruction that can access memory can be used to transfer data to or from an I/O device.

eg:

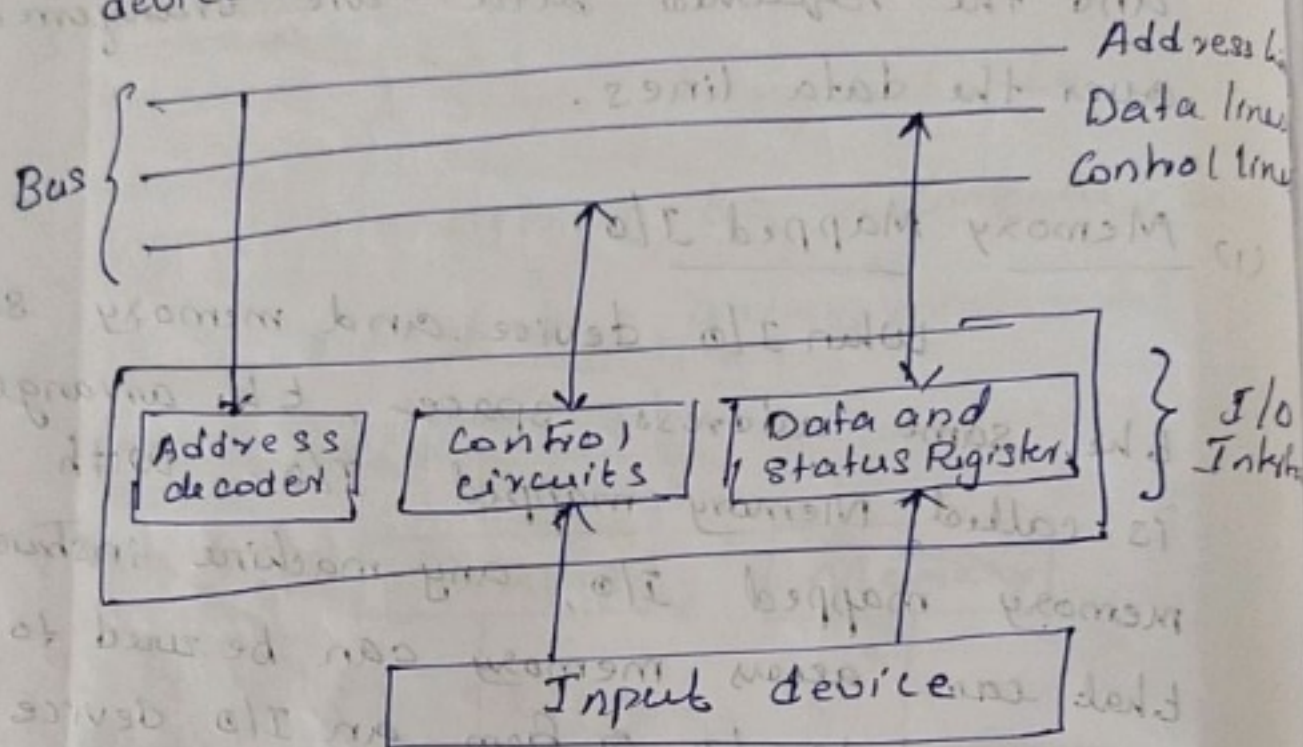
If `DATAIN` is the address of the input buffer associated with the keyboard, the instruction,

```
MOV DATAIN, R0
```

reads the data from `DATAIN` and stores it in processor register `R0`.

Most computer systems use memory-mapped I/O. Some processors have special in and out instructions to perform I/O transfers.

The below figure illustrates the hardware required to connect an I/O device to the bus.



The Address decoder enables the device to recognize its address when this address appears on the address lines. The data register holds the data being transferred to or from the processor. The status register contains information relevant to the operation of the I/O device. Both the data and status registers are connected to the

data bus and assigned unique addresses. The address decoder, control circuits and data and status registers are required to coordinate I/O transfers, constitute the ~~devices~~, I/O Interface Circuit

(2) Program Controlled I/O

In program-controlled I/O, the ~~prog~~ processor program controls the complete data transfer. So only when an I/O transfer instruction is executed, the transfer could take place. It is required to check that device is ready/not for the data transfer. Extra hardware is not required. Processor issues a command to the I/O module and waits for the operation to complete. Also, the processor keeps checking the I/O module status until it finds the completion of the operation.

There are two commonly used mechanisms for implementing I/O operation

- (1) Interrupt
- (2) Direct Memory Access.

In case of Interrupts, synchronization is achieved by having the I/O device send a special signal over the bus, whenever it is ready for a data transfer.

DMA is a technique used for high-speed I/O devices. It involves having the device interface transfer data directly to or from the memory, without continuous involvement by the processor.

II Interrupts

When a program enters a wait loop in which it repeatedly tests the device status. During this period, the processor is not performing any computation.

There are many situations, where other tasks can be performed while waiting for an I/O device to become ready. To allow this to happen, we can arrange for the I/O device to alert the processor when it becomes ready. It can be done by

Sending a hardware signal called an interrupt to the processor. At least one of the bus control line, interrupt-request line, is dedicated for this purpose. The routine executed in response to ~~the~~ an interrupt request is called the interrupt-service routine.

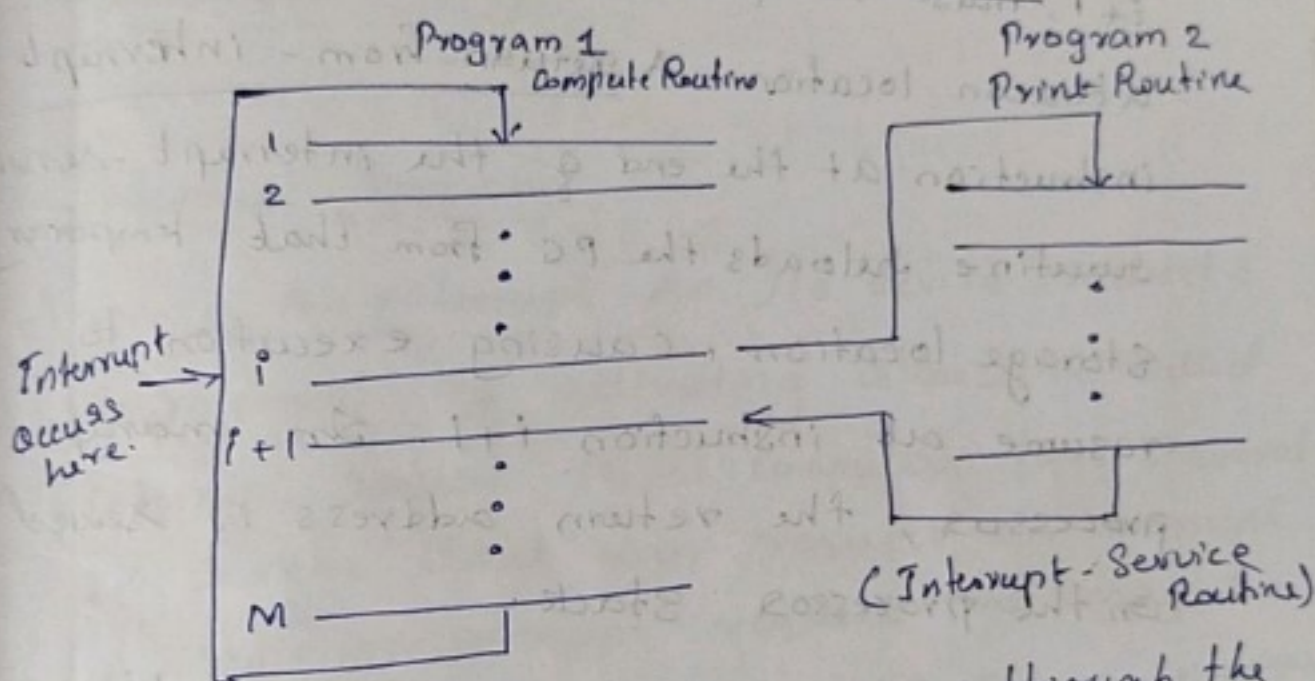


fig: Transfer of Control through the use of Interrupts.

Assume that an interrupt request arrives during execution of instruction 'i' in the above figure. The processor first completes execution of instruction 'i'. Then, it loads the program counter with the address of the first instruction of the Interrupt-Service

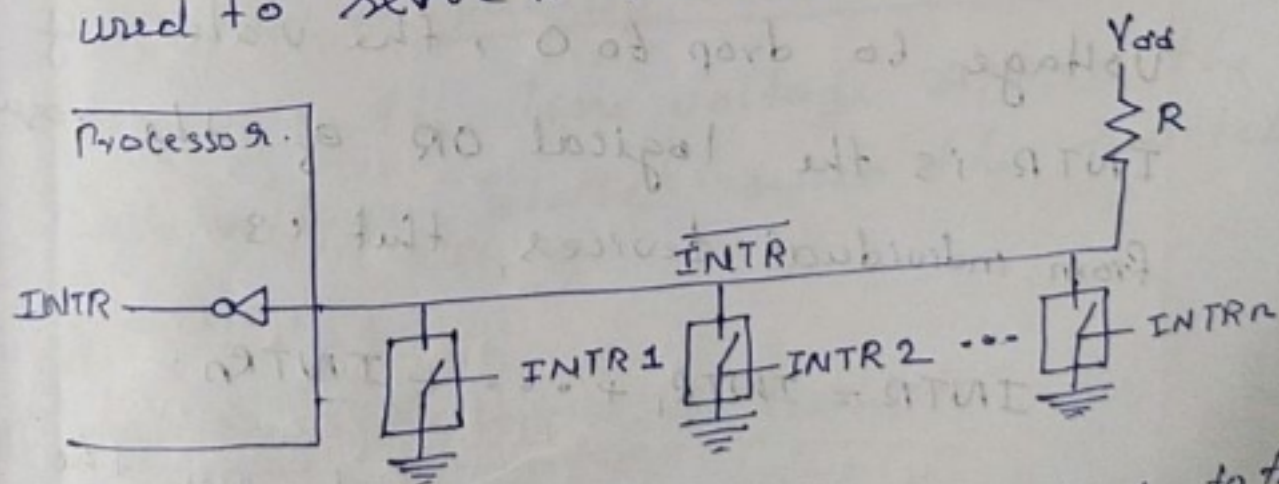
routine (here print routine). After execution of the interrupt-service routine, the processor has to come back to instruction $i+1$. Therefore, when an interrupt occurs, the current contents of the PC, which point to instruction $i+1$, must be put in temporary storage in a known location. A return-from-interrupt instruction at the end of the interrupt-service routine reloads the PC from that temporary storage location, causing execution to resume at instruction $i+1$. In many processors, the return address is saved on the processor stack.

While handling interrupts, the processor must inform the device that its request has been recognised, so that it may remove its interrupt-request signal. This is done by special control signal called Interrupt Acknowledge signal. The process of saving and restoring registers involves memory

transfers, that increases the total execution time. Saving registers also increase the delay between the time an interrupt request is received and the start of execution of the interrupt-service routine. This delay is called Interrupt latency.

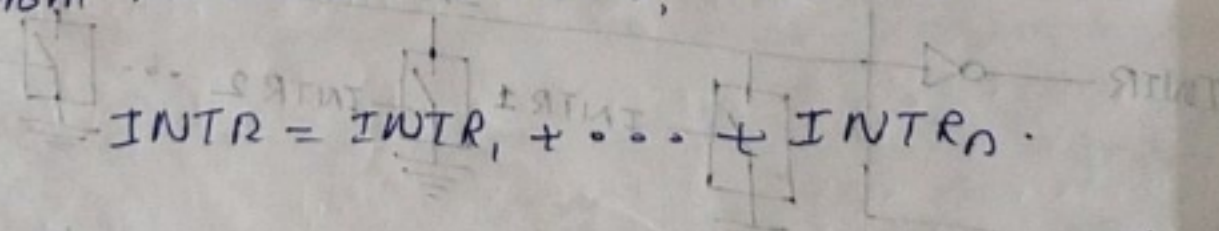
Interrupt Hardware.

~~An interrupt~~ An I/O device requests an interrupt by activating a bus line called interrupt-request. Most computers have several I/O devices that can request an interrupt. A single interrupt request line may be used to serve 'n' devices as shown in figure.



All devices are connected to the line via switches to ground. To request an interrupt, a device closes its switch.

Thus, if all interrupt-request signals $INTR_1$ to $INTR_n$ are inactive, if all the switches are open, the voltage on the interrupt-request line will be equal to V_{dd} . That is the inactive state of the line. When a device requests an interrupt by closing its switch, the voltage on the line drops to 0, causing the interrupt-request signal, $INTR$, received by the processor to go to 1. Since the closing of one or more switches will cause the line voltage to drop to 0, the value of $INTR$ is the logical OR of the request from individual devices, that is


$$INTR = INTR_1 + \dots + INTR_n.$$

It is customary, to use the complemented form, $\overline{INTR}$, to name the interrupt-request signal on the

common line, because this signal is active when it is in the low-voltage state.

In this electronic implementation, of the circuit in figure, special gates known as open-collector (for bipolar circuits) or open drain (for MOS circuits) are used to drive the $\overline{INTA}$ line. The output of an open-collector or an open-drain gate is equivalent to a switch to ground that is open, when the gate's input is in the 0 state and closed when it is in the 1 state. Resistor R is called a pull-up resistance, because it pulls the line voltage up to the high-voltage state when the switches are open.

Handling Multiple Interrupts

Consider the situation where a number of devices capable of initiating interrupts are connected to the processor.

This gives rise to a no. of questions:

(1) How can the processor recognize the device requesting an interrupt?

(2) How the processor, identifies and obtain the starting address of the appropriate routine in each case?

(3) How two or more simultaneous interrupts are handled? etc.

These problems are resolved, which may vary from one computer to another.

When a request is received over the common interrupt-request line, an additional

information is needed to identify the particular device. If two devices ^{have} ~~are~~

activated the line at the same time, select

one of the two request for service.

When the interrupt service routine for the selected device is completed, the second request is serviced.

The information needed to determine whether a device is requesting an interrupt is available in its status register. When a device raises an interrupt request, it sets to 1, one of the bits in its status register, which is IRQ bit. The simplest way to identify the interrupting device is to have the interrupt-service routine poll all the I/O devices connected to the devices. The first device encountered with its IRQ bit set is the device that should be serviced. An appropriate subroutine is called to provide the requested service.

Vectored Interrupt

An alternative approach to polling process is the vectored interrupt. To reduce the time involved in polling process, a device requesting an interrupt may identify itself directly to the processor.

Then, the processor can immediately start executing the corresponding interrupt-service routine. The term vectored interrupt refers to all interrupt handling schemes based on this approach.

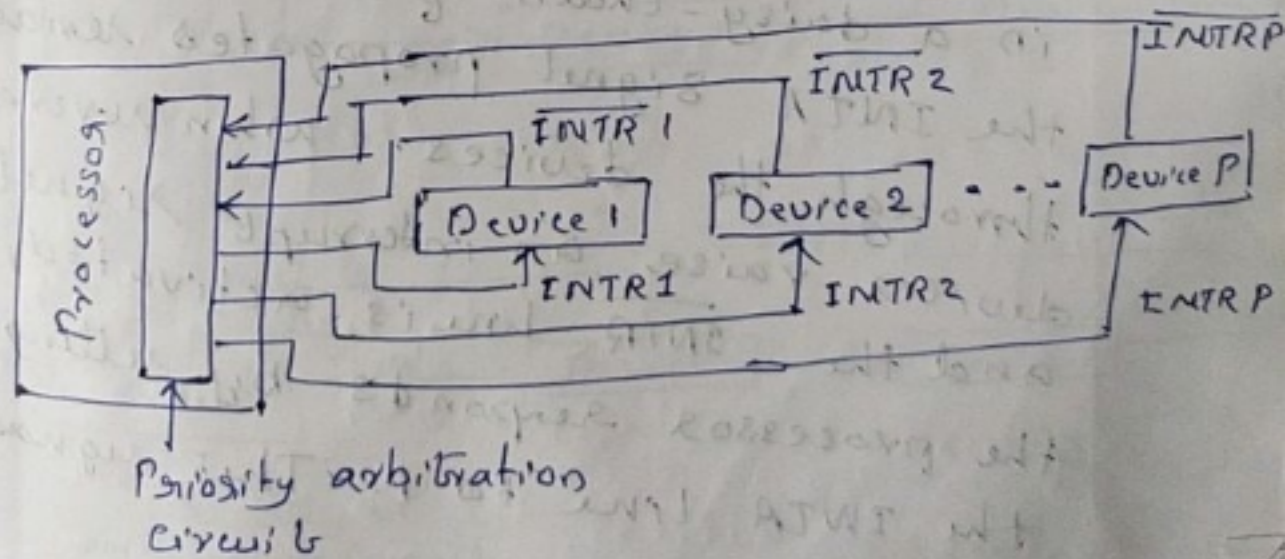
A device requesting an interrupt can identify itself by sending a special code to the processor over the bus. This enables the processor to identify individual devices even if they share a single interrupt-request line. The code supplied by the device may represent the starting address of the interrupt service routine for that device.

The code length is typically in the range of 4 to 8 bits. The remaining of address is supplied by the processor based on the area in its memory, where the addresses for interrupt-service routines are located.

The location pointed to by the interrupting device is used to store the starting address of the interrupt service routine. The processor reads this address called the Interrupt Vector and loads it into the PC. The Interrupt Vector may also include a new value for the processor status register.

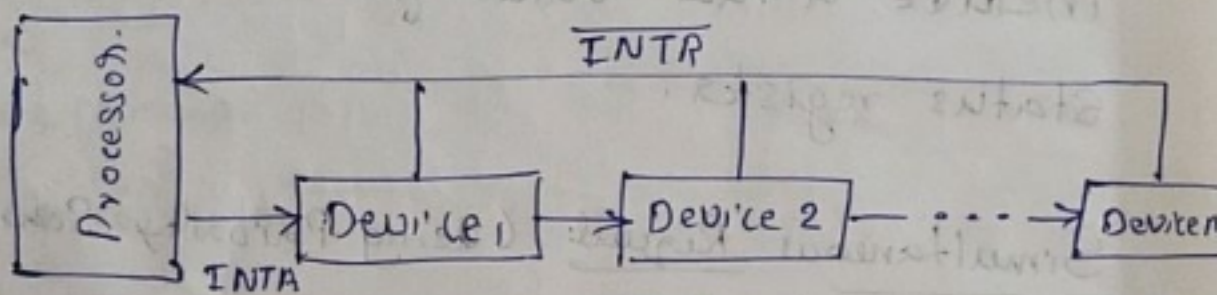
Simultaneous Request (Using Priority Scheme).

When simultaneous request arrives, it can be handled by using priority scheme, as shown in figure below.



The processor simply accepts the request having the highest priority, i.e. several devices share one interrupt - request line.

Another scheme is to connect the devices to form a daisy chain, as shown in figure.



The Interrupt request line is common to all devices. The Interrupt acknowledge line, INTA, is connected in a daisy-chain fashion, such that the INTA signal propagates serially through the devices. When several devices raise an interrupt signal and the INTR line is activated, the processor responds by setting the INTA line to 1. This signal

is serviced by device 1. Device 1 passes the signal on to device 2 only, if it does not require any service. If device 1 has a pending request for interrupt, it blocks the INTA signal and proceeds to put its identifying code on the data line. Thus the device electrically closest to the processor have the highest priority and the second device with second highest priority and so on.

These two schemes can be

combined to produce the more general structure shown below. This is used in many computer systems.

